



M Ű E G Y E T E M 1 7 8 2

# MASTER THESIS

Development and implementation of  
polynomial algorithms for computations  
involving large tensor powers of matrices

Martin Áron Juhász

Supervisor: Dr. Mihály Weiner  
Department of Mathematical Analysis  
Institute of Mathematics  
Budapest University of Technology and Economics (BME)

Budapest University of Technology and Economics

2026



## Disclosure

I, the undersigned, a student of the Budapest University of Technology and Economics, studying Applied Mathematics MSc, declare that I have prepared this thesis independently, without unauthorized assistance, under the guidance of my supervisor, and have only used the specified sources. I have indicated with proper citation every part that I have directly quoted or paraphrased from other sources.

Pisa, Italy, June 4, 2026

---

**Martin Áron Juhász**

# Contents

|          |                                                                                                 |           |
|----------|-------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                             | <b>1</b>  |
| <b>2</b> | <b>Theoretical Background: Block Decomposition of Tensor Powers via Virtual Representations</b> | <b>4</b>  |
| 2.1      | The Dimensionality of Tensor Powers and Schur-Weyl Duality . . . . .                            | 4         |
| 2.2      | The Symmetric Algebra and Computable Representations . . . . .                                  | 6         |
| 2.3      | Theoretical Background: Generating the Symmetric Representation . . .                           | 6         |
| 2.3.1    | Orthonormal Basis Construction for $\text{Sym}^k(\mathbb{C}^d)$ . . . . .                       | 6         |
| 2.3.2    | Direct Multinomial Expansion of Matrix Elements . . . . .                                       | 7         |
| 2.4      | Factorization of the Determinant Representation for $\text{GL}(3, \mathbb{C})$ . . . . .        | 8         |
| 2.5      | The Limits of the Semi-Ring: Exploring the $n = 4$ and $n = 5$ Cases . . .                      | 9         |
| 2.6      | The Representation Ring and the Jacobi-Trudi Identity . . . . .                                 | 11        |
| 2.7      | Application to Schatten $p$ -Norms . . . . .                                                    | 13        |
| <b>3</b> | <b>Examples</b>                                                                                 | <b>15</b> |
| 3.1      | Illustrative Examples: Tensor Product Decompositions . . . . .                                  | 15        |
| 3.1.1    | Combinatorial Evaluation of Target Space via Hook and Weyl Formulas . . . . .                   | 15        |
| 3.1.2    | The $n = 4$ Case: Exact Deterministic Decomposition via Jacobi-Trudi . . . . .                  | 16        |
| 3.2      | Explicit Evaluation of the Multinomial Expansion . . . . .                                      | 17        |
| <b>4</b> | <b>Computational Methodology and Software Architecture</b>                                      | <b>19</b> |
| 4.1      | High-Level Algorithmic Pipeline . . . . .                                                       | 19        |
| 4.2      | Data Architecture: Sparse Tensor Compression . . . . .                                          | 20        |
| 4.3      | Runtime Vectorization and Parallelization . . . . .                                             | 21        |
| 4.4      | Algorithmic Efficiency of the Jacobi-Trudi Decomposition . . . . .                              | 21        |
| 4.5      | Alternative Approach and Future Work: Integer Linear Programming . .                            | 22        |
| <b>5</b> | <b>Results</b>                                                                                  | <b>24</b> |
| 5.1      | Dimensionality and Memory Space Scaling . . . . .                                               | 24        |
| 5.2      | Computational Runtime Scaling . . . . .                                                         | 27        |
| <b>6</b> | <b>Software Implementation and Open-Source Package</b>                                          | <b>30</b> |
| 6.1      | Package Capabilities and Storage Constraints . . . . .                                          | 30        |
| 6.2      | Integration of $2 \times 2$ Matrix Decompositions . . . . .                                     | 30        |
| 6.3      | Concurrency and Parallelization Strategy . . . . .                                              | 31        |

|                         |           |
|-------------------------|-----------|
| <b>7 Conclusion</b>     | <b>32</b> |
| <b>Acknowledgements</b> | <b>33</b> |

# 1 Introduction

In many areas of mathematics and physics, particularly within quantum information theory and statistical mechanics, one encounters expressions involving high tensor powers of matrices. A prominent example is the evaluation of quantities of the form

$$\left\| \sum_i t_i A_i^{\otimes n} \right\|_p, \quad (1)$$

where  $A_1, A_2, \dots \in M_d(\mathbb{C})$  are fixed matrices,  $t_1, t_2, \dots \in \mathbb{C}$  are fixed coefficients, and  $\|\cdot\|_p$  denotes the Schatten  $p$ -norm.

A primary physical motivation for evaluating such expressions arises from quantum hypothesis testing. Suppose an observer is given a quantum system and promised it is prepared in either state  $\rho$  (with prior probability  $p_0$ ) or state  $\sigma$  (with prior probability  $p_1$ ). To optimally distinguish between these states given  $n$  independent and identically distributed copies, one performs asymptotic binary hypothesis testing:

$$\begin{aligned} H_0 &: \text{The state is } \rho^{\otimes n} \\ H_1 &: \text{The state is } \sigma^{\otimes n} \end{aligned}$$

According to the Helstrom-Holevo theorem [1], [2], the minimum error probability  $P_{\text{err}}(n)$  achievable by any physically permissible quantum measurement (Positive Operator-Valued Measure, or POVM) is given exactly by the trace norm (Schatten 1-norm)[3]:

$$P_{\text{err}}(n) = \frac{1}{2} \left( 1 - \|p_0 \rho^{\otimes n} - p_1 \sigma^{\otimes n}\|_1 \right) \quad (2)$$

Furthermore, this classification task naturally generalizes to multiple hypothesis testing, where an observer must distinguish among an arbitrary number of candidate states  $\{\rho_1^{\otimes n}, \dots, \rho_m^{\otimes n}\}$  [4].

While the mathematical formulation of these problems is clear, evaluating them analytically poses a challenge that scales drastically with the number of matrices involved. For a single matrix tensor power, quantities such as the Schatten  $p$ -norm and the determinant are given trivially by explicit closed-form formulas (e.g.,  $\|A^{\otimes n}\|_p = \|A\|_p^n$ ).

However, for a linear combination of the  $n$ -th tensor powers of two matrices, analytical solutions become restricted. For the Schatten 1-norm (the trace norm), no exact algebraic formula exists; analytical results are limited strictly to asymptotic bounds and limits as  $n \rightarrow \infty$ . Conversely, the determinant of a two-matrix combination remains an exception, as it can still be evaluated explicitly for any finite  $n$ .

Crucially, when the problem expands to linear combinations of three or more matrices, the exact regime required for multiple hypothesis testing, the analytical landscape breaks

down entirely. For three or more matrices, there are no known asymptotic limits for the Schatten norms, nor are there explicit algebraic formulas for the determinant.

Because analytical formulas and limits are unavailable for these complex combinations, one must resort to direct numerical calculation. However, evaluating these expressions directly requires computing the norm of a matrix acting on a space of dimension  $d^n$ . This leads to an exponential growth in computational complexity in the tensor power parameter  $n$ , rendering direct algebraic approaches impossible for large tensor powers. The purpose of the present work is to develop and implement a representation-theoretic framework which reduces such computations from exponential to polynomial complexity, bypassing the exponential space entirely. The key observation is that the tensor power representation  $U \mapsto U^{\otimes n}$  of  $\mathrm{SU}(d)$  admits a decomposition into isotypic components:

$$WU^{\otimes n}W^* = \bigoplus_j (I_{m_j} \otimes \pi_j(U)), \quad (3)$$

where the  $\pi_j$  are irreducible representations and the  $m_j$  are multiplicities. Since unitary similarity preserves all unitarily invariant quantities, computations involving massive tensor powers reduce to computations on substantially smaller blocks.

The irreducible representations appearing in this decomposition extend naturally from  $\mathrm{SU}(d)$  to polynomial representations of  $\mathrm{GL}(d, \mathbb{C})$ . Consequently, for every matrix  $A \in M_d(\mathbb{C})$ , one obtains a structural block decomposition:

$$WA^{\otimes n}W^* = \bigoplus_j (I_{m_j} \otimes \hat{\pi}_{n,j}(A)), \quad (4)$$

where the matrices  $\hat{\pi}_{n,j}(A)$  have dimensions growing only polynomially in  $n$ .

Crucially, because this decomposition is a structural property of the tensor space governed by the diagonal action of the linear group, the basis transformation  $W$  is universal and entirely independent of the specific matrix  $A$ . It is this shared, common basis that allows linear combinations of massive tensor products to remain perfectly block-diagonal. This universal isomorphism immediately yields computationally tractable formulas such as:

$$\left\| \sum_i t_i A_i^{\otimes n} \right\|_p^p = \sum_j m_j \left\| \sum_i t_i \hat{\pi}_{n,j}(A_i) \right\|_p^p, \quad (5)$$

alongside analogous identities for determinants, spectra, and other unitarily invariant quantities. A central practical difficulty is that explicit matrix realizations of general irreducible polynomial representations are rarely available in the literature. While symmetric and exterior powers admit straightforward concrete constructions, arbitrary irreducible representations typically appear only at the infinitesimal level through Lie algebra generators or highest-weight theory. For computational purposes, however, a full irreducible

decomposition is not strictly necessary; the essential computational advantage stems from factoring out the large multiplicities  $m_j$  rather than achieving maximal irreducibility.

Motivated by this observation, we introduce a more flexible framework based on a restricted family of explicitly computable polynomial representations. In the  $d = 2$  case, every irreducible representation is simply a symmetric power representation (up to a determinant twist), yielding exact irreducible decompositions. For  $d = 3$ , we show that every polynomial representation belongs, within the Grothendieck ring, to the additive subgroup generated by tensor products of at most two elementary representations—where the elementary representations consist of symmetric powers, exterior powers, and their determinant twists. More generally, we prove that for arbitrary  $d$ , tensor products of at most  $d - 1$  such elementary representations additively generate the entire polynomial representation ring.

This representation-theoretic reduction leads to highly efficient numerical algorithms. We implemented the resulting methods in a software framework capable of handling tensor powers far beyond the reach of brute-force methods. For example, evaluating tensor powers of order  $n = 50$  for  $2 \times 2$  matrices can be processed within seconds on standard hardware.

The present thesis develops the theoretical foundations underlying this approach. In subsequent sections, we discuss the representation-theoretic background, explicit polynomial realizations of the required representations, virtual decompositions within the Grothendieck ring, algorithmic implementation details, and concrete examples illustrating the computational gain over direct tensor-product methodologies.



## 2 Theoretical Background: Block Decomposition of Tensor Powers via Virtual Representations

### 2.1 The Dimensionality of Tensor Powers and Schur-Weyl Duality

Let  $V$  be a finite-dimensional complex vector space, and let  $M$  be an operator acting on  $V$ . The  $n$ -fold tensor power of  $V$ , denoted  $V^{\otimes n}$ , scales exponentially in dimension, rendering direct numerical procedures computationally infeasible for large  $n$ . However, when restricted to the  $n$ -fold tensor power of a single operator,  $M^{\otimes n}$ , which corresponds to the diagonal action of the linear group, the tensor space exhibits a highly reducible invariant structure. As developed in classical works on representation theory, this specific reducibility is governed by the commuting actions of the continuous linear group and the discrete symmetric group [5], [6].

**Theorem 1** (Schur-Weyl Duality [5], [6]). *Let  $V = \mathbb{C}^d$  be a finite-dimensional complex vector space, and consider the  $n$ -fold tensor power  $V^{\otimes n}$ . There are two natural, commuting group actions on this space: the general linear group  $GL(V)$  acts diagonally from the left via*

$$g(v_1 \otimes \cdots \otimes v_n) = gv_1 \otimes \cdots \otimes gv_n, \quad g \in GL(V),$$

*and the symmetric group  $S_n$  acts by permuting the tensor factors from the right,*

$$(v_1 \otimes \cdots \otimes v_n)\sigma = v_{\sigma(1)} \otimes \cdots \otimes v_{\sigma(n)}, \quad \sigma \in S_n.$$

*The fundamental assertion of Schur-Weyl duality is that the algebras generated by these two actions are mutual centralizers within  $\text{End}(V^{\otimes n})$ . Consequently, under the joint action of  $GL(V) \times S_n$ , the tensor space admits a multiplicity-free decomposition into mutually determining irreducible representations:*

$$V^{\otimes n} \cong \bigoplus_D \mathbb{S}_D(V) \otimes S^D.$$

*Here, the direct sum ranges over all Young diagrams  $D$  with  $n$  boxes and at most  $d$  rows. The space  $S^D$  denotes the irreducible representation of  $S_n$  (the Specht module) associated to the diagram  $D$ , and  $\mathbb{S}_D(V)$  is the corresponding irreducible polynomial representation of  $GL(V)$  (the Weyl/Schur module).*

**Application to  $SU(3)$ :** In our computational framework, we specialize to  $V = \mathbb{C}^3$  ( $d = 3$ ) and restrict the continuous group from  $GL(3, \mathbb{C})$  to the special unitary group  $SU(3)$ .

Because  $SU(3)$  is a compact real form of  $SL(3, \mathbb{C})$ , the restriction of the Weyl modules  $\mathbb{S}_D(\mathbb{C}^3)$  to  $SU(3)$  remains completely irreducible. Viewed strictly as an  $SU(3)$  module, the tensor space decomposition specializes directly to our domain:

$$(\mathbb{C}^3)^{\otimes n} \cong \bigoplus_D \left( V_D^{SU(3)} \right)^{\oplus f^D} \quad (6)$$

The dimension of the Specht module,  $f^D = \dim(S^D)$ , dictates the multiplicity of the irreducible representation  $V_D^{SU(3)}$  in the tensor power. While the classical Hook-length formula (see Theorem 2) defines this as  $f^D = n! / \prod_{c \in D} h_c$  (where  $h_c$  is the hook length of cell  $c$ ), our computational domain is strictly bounded to partitions of at most three rows,  $D = (\lambda_1, \lambda_2, \lambda_3)$ .

For this specific case, the product of the hook lengths can be resolved into an explicit, closed-form algebraic equation:

$$f^D = \frac{n! (\lambda_1 - \lambda_2 + 1)(\lambda_1 - \lambda_3 + 2)(\lambda_2 - \lambda_3 + 1)}{(\lambda_1 + 2)! (\lambda_2 + 1)! \lambda_3!} \quad (7)$$

where  $n = \lambda_1 + \lambda_2 + \lambda_3$  [5].

To formalize the general combinatorial principle from which this explicit 3-row derivation originates, we define the hook length and the associated dimensional formula for an arbitrary partition:

**Theorem 2** (The Hook Length Formula [5]). *Let  $D$  be a Young diagram corresponding to a partition  $\lambda \vdash n$ . For each cell  $c = (i, j) \in D$  (located in row  $i$  and column  $j$ ), the hook length  $h_c$  is defined as the number of cells in the same row strictly to the right of  $c$ , plus the number of cells in the same column strictly below  $c$ , plus 1 for the cell  $c$  itself. Mathematically, this is expressed as:*

$$h_{i,j} = (\lambda_i - j) + (\lambda'_j - i) + 1$$

where  $\lambda'$  denotes the conjugate partition (the transposed Young diagram). The dimension  $f^D$  of the corresponding irreducible representation (Specht module) of the symmetric group  $S_n$  is exactly:

$$f^D = \frac{n!}{\prod_{c \in D} h_c} \quad (8)$$

**Remark (Purely Symmetric Partitions):** Consider the completely symmetric representation corresponding to a single row of  $n$  boxes, denoted by the partition  $\lambda = (n)$ . The hook lengths of the cells from left to right are strictly  $n, n-1, \dots, 1$ , yielding a product of exactly  $n!$ . By Theorem 2, the dimension of the associated Specht module is  $f^{(n)} = \frac{n!}{n!} = 1$ . This mathematically formalizes the fact that the completely symmetric

subspace  $\text{Sym}^n(V)$  appears with a multiplicity of exactly 1 within the full tensor power  $V^{\otimes n}$ . (While its multiplicity is 1, the internal spatial dimension of this block for  $\text{SU}(3)$  evaluates to  $\frac{1}{2}(n+1)(n+2)$  via the Weyl Dimension Formula, see Equation 16).

**Remark (Antisymmetric Partitions and  $\Lambda^2$ ):** Similarly, for a completely antisymmetric representation defined by a single column of  $k$  boxes (the partition  $\lambda = (1^k)$ ), the vertical hook lengths are  $k, k-1, \dots, 1$ . This again yields a product of  $k!$  and a theoretical multiplicity of  $f^{(1^k)} = \frac{k!}{k!} = 1$  within  $V^{\otimes k}$ . In the context of our foundational computational basis, the relevant antisymmetric exterior power  $\Lambda^2(\mathbb{C}^3)$  corresponds to the partition  $\lambda = (1, 1)$ . The product of its hook lengths is  $2 \times 1 = 2$ , verifying its multiplicity is  $f^{(1,1)} = \frac{2!}{2} = 1$  in  $V^{\otimes 2}$ . (By the Weyl Dimension Formula, the internal spatial dimension of this  $\Lambda^2$  block in  $\text{SU}(3)$  is 3).

## 2.2 The Symmetric Algebra and Computable Representations

While arbitrary highest-weight representations are structurally well-understood, constructing explicit matrix representations for them is difficult. We therefore restrict our foundational computational building blocks to a specific, computable class of representations.

**Definition 1** (Symmetric Representation [5]). *The  $k$ -th symmetric power of a vector space  $V$ , denoted  $\text{Sym}^k(V)$ , is defined as the maximal commutative quotient of the tensor algebra. Specifically, it is the quotient of  $V^{\otimes k}$  by the ideal generated by all alternating elements of the form  $v \otimes w - w \otimes v$ . It is naturally isomorphic to the space of homogeneous polynomials of degree  $k$  in  $d$  variables.*

The matrix elements of  $\pi_{\text{Sym}^k}(M)$  can be evaluated analytically and efficiently without iteratively constructing highest-weight vectors. By employing a standard symmetric multinomial expansion, the basis vectors of  $\text{Sym}^k(\mathbb{C}^d)$  are fully characterized by multi-indices  $\alpha = (\alpha_1, \dots, \alpha_d)$  such that  $\sum \alpha_i = k$ , yielding exact combinatorial normalization coefficients that inherently preserve unitarity.

## 2.3 Theoretical Background: Generating the Symmetric Representation

### 2.3.1 Orthonormal Basis Construction for $\text{Sym}^k(\mathbb{C}^d)$

Consider the  $k$ -th symmetric power of a  $d$ -dimensional vector space, denoted as  $\text{Sym}^k(\mathbb{C}^d)$ . In the context of Schur-Weyl duality, this corresponds to the Weyl module associated with the completely symmetric partition consisting of a single row of  $k$  boxes.

A natural basis for this symmetric subspace is indexed by weak compositions of  $k$  into  $d$  parts. We represent each basis vector by a tuple of non-negative integers  $\mathbf{v} = (v_1, v_2, \dots, v_d)$  such that  $\sum_{i=1}^d v_i = k$ . Each integer  $v_i$  dictates the multiplicity of the  $i$ -th standard basis vector  $e_i$  in the symmetric product.

To ensure the resulting matrix representations are unitary when evaluated on unitary matrices, the basis must be orthonormalized with respect to the standard inner product on the full tensor space. The normalized basis state  $|\mathbf{v}\rangle$  is scaled by a factor  $\mathcal{N}(\mathbf{v})$  derived from the multinomial distribution:

$$\mathcal{N}(\mathbf{v}) = \sqrt{\frac{v_1! v_2! \dots v_d!}{k!}} \quad (9)$$

### 2.3.2 Direct Multinomial Expansion of Matrix Elements

The matrix elements of the representation  $\pi_{\text{Sym}^k}(M)$  for an arbitrary matrix  $M \in \text{GL}(d)$  can be evaluated directly by computing the exact transition amplitudes between the orthonormal symmetric basis vectors.

Let  $|\mathbf{c}\rangle = (c_1, \dots, c_d)$  denote the source (column) basis state and  $|\mathbf{r}\rangle = (r_1, \dots, r_d)$  denote the target (row) basis state. The transition maps the basis elements according to the entries of  $M$ . We define a  $d \times d$  integer exponent matrix  $E$  where each entry  $e_{u,v}$  represents the exact number of times the basis vector  $e_v$  transitions to  $e_u$  via the specific matrix element  $M_{u,v}$ .

For a given configuration  $E$  to be valid, the sum of its entries must satisfy the source and target constraints:

$$\sum_{v=1}^d e_{u,v} = r_u \quad (\text{row constraints}) \quad (10)$$

$$\sum_{u=1}^d e_{u,v} = c_v \quad (\text{column constraints}) \quad (11)$$

Because the operator acts on  $k$  independent factors simultaneously, the combinatorial number of ways this specific transition configuration  $E$  can occur across the  $k$  tensor slots is given by the multinomial coefficient:

$$C(E) = \frac{k!}{\prod_{u,v} e_{u,v}!} \quad (12)$$

The overall matrix element  $\langle \mathbf{r} | \pi_{\text{Sym}^k}(M) | \mathbf{c} \rangle$  is obtained by summing over all valid exponent matrices  $E$ . For each valid  $E$ , the combinatorial coefficient  $C(E)$  is multiplied by the normalizations of both the row and column states, and the corresponding product of the matrix elements  $M_{u,v}$  raised to the power of  $e_{u,v}$ :

$$\langle \mathbf{r} | \pi_{\text{Sym}^k}(M) | \mathbf{c} \rangle = \sum_E C(E) \cdot \mathcal{N}(\mathbf{r}) \cdot \mathcal{N}(\mathbf{c}) \prod_{u,v} M_{u,v}^{e_{u,v}} \quad (13)$$

Substituting the definitions for the normalizations, the  $k!$  from the multinomial coefficient and the  $\sqrt{k!}$  terms from the two normalizations cancel out. This simplifies to a closed-form equation:

$$\langle \mathbf{r} | \pi_{\text{Sym}^k}(M) | \mathbf{c} \rangle = \sum_E \left( \frac{\sqrt{\prod_{i=1}^d r_i! \prod_{j=1}^d c_j!}}{\prod_{u,v} e_{u,v}!} \right) \prod_{u,v} M_{u,v}^{e_{u,v}} \quad (14)$$

This closed-form expansion allows for the direct theoretical formulation of the matrix elements of the symmetric power representation entirely through combinatorial properties, without necessitating the construction or consideration of the full tensor space. Mathematically, this explicit element-wise construction corresponds to evaluating the inner product of symmetric tensors via the permanent of their associated Gram matrix, a fundamental property formalized in combinatorial matrix theory [7, Theorem 2.2]. By applying this property to an orthonormal basis, the transition amplitudes reduce exactly to normalized permanents of matrices with repeated rows and columns, yielding the explicit summation over transition configurations  $E$  derived above.

**Remark (Physical Interpretation).** This combinatorial construction maps directly to the quantum mechanics of indistinguishable bosons. The symmetric tensor space  $\text{Sym}^k(\mathbb{C}^d)$  represents the  $k$ -particle sector of a bosonic Fock space spanning  $d$  modes, where the integer tuples  $\mathbf{c}$  and  $\mathbf{r}$  define the initial and final Fock states (occupation numbers). If  $M$  is a unitary scattering matrix modeling a linear optical network, this closed-form expansion computes the exact multi-particle transition amplitude from state  $|\mathbf{c}\rangle$  to  $|\mathbf{r}\rangle$  [8].

## 2.4 Factorization of the Determinant Representation for $\text{GL}(3, \mathbb{C})$

While our basis generation relies on the algebra of  $\text{SU}(3)$ , the computational framework is designed to process arbitrary  $3 \times 3$  matrices. This requires working within the representation theory of the general linear group,  $\text{GL}(3, \mathbb{C})$ .

**Definition 2** (Determinant Representation [5]). *For the fundamental representation  $V = \mathbb{C}^d$ , the highest non-vanishing exterior power  $\Lambda^d(V)$  is a one-dimensional representation corresponding to a single column of  $d$  boxes in a Young diagram. For any operator  $M \in \text{GL}(V)$ , this space transforms by the scalar factor  $v \mapsto \det(M)v$ .*

Because we are working with  $d = 3$ , any full column of length three in a Young diagram  $D$  corresponds geometrically to twisting the representation by the determinant. Representing the diagram by its row lengths as an integer partition  $\lambda = (\lambda_1, \lambda_2, \lambda_3)$ , we

can factor any arbitrary representation of  $GL(3, \mathbb{C})$  into a “reduced”  $SU(3)$ -like representation tensored with a power of the determinant [5]:

$$V_{(\lambda_1, \lambda_2, \lambda_3)} \cong V_{(\lambda_1 - \lambda_3, \lambda_2 - \lambda_3, 0)} \otimes \det(M)^{\lambda_3} \quad (15)$$

Notice that the partition on the right-hand side, representing the absolute row lengths, naturally yields the standard Dynkin labels  $l_1 = (\lambda_1 - \lambda_3) - (\lambda_2 - \lambda_3) = \lambda_1 - \lambda_2$  and  $l_2 = (\lambda_2 - \lambda_3) - 0 = \lambda_2 - \lambda_3$ .

Computationally, this implies that any full column of length three can be structurally truncated from the Young diagram, provided we track the number of columns removed,  $m = \lambda_3$ . Every irreducible block generated in our tensor decomposition is uniquely characterized by exactly three integers: the two Dynkin labels  $l_1 = \lambda_1 - \lambda_2$  and  $l_2 = \lambda_2 - \lambda_3$  (which dictate the internal matrix structure), and the determinant power  $m$  (which acts as a global scalar multiplier).

**Dimensionality of the Reduced Representation:** While the Hook-Length formula (Theorem 2) dictates the multiplicity of an invariant block within the tensor space, its internal spatial dimension is determined by the Weyl Dimension Formula. By evaluating the general dimension formula [5, Eq. 15.17] specifically for  $n = 3$ , the dimension of the module reduces to a closed-form symmetric polynomial dependent strictly on its Dynkin labels  $l_1$  and  $l_2$ :

$$\dim \left( V_{(l_1, l_2)}^{\text{SU}(3)} \right) = \frac{1}{2} (l_1 + 1)(l_2 + 1)(l_1 + l_2 + 2) \quad (16)$$

**Note on the Determinant Representation:** Because the determinant representation is strictly one-dimensional, algebraically factoring it out (by actively truncating full columns from the Young diagram) does not alter the spatial dimension of the module. Since tensoring with a one-dimensional space preserves dimensionality, the spatial dimension of any resulting block is governed entirely by this evaluated polynomial.

## 2.5 The Limits of the Semi-Ring: Exploring the $n = 4$ and $n = 5$ Cases

Before formalizing the algebraic rules of our decomposition, it is instructive to observe why physical direct sums (the semi-ring of representations) are insufficient when restricting our building blocks to a specific, computable pool. Suppose we attempt to reconstruct the target tensor space using linear combinations of 2-term symmetric and antisymmetric tensor products.

**The  $n = 4$  Case: A Strictly Positive Solution** For the fourth tensor power, the 81-dimensional space decomposes into the target irreducible representations:

$$(\square)^{\otimes 4} \cong 1 \square\square\square\square \oplus 3 \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & & \\ \hline \end{array} \oplus 2 \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \oplus 3 \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \\ \hline \end{array}$$

By evaluating the unique combinations of symmetric and antisymmetric products that sum to  $n = 4$  boxes, we can generate a candidate pool of computable blocks:

- **A** ( $\text{Sym}^1 \otimes \text{Sym}^3$ ):  $\square \otimes \square\square\square \cong \square\square\square\square \oplus \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & & \\ \hline \end{array}$
- **B** ( $\text{Sym}^1 \otimes \Lambda^3$ ):  $\square \otimes \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} \cong \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \\ \hline \end{array}$
- **C** ( $\text{Sym}^2 \otimes \text{Sym}^2$ ):  $\square\square \otimes \square\square \cong \square\square\square\square \oplus \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & & \\ \hline \end{array} \oplus \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array}$
- **D** ( $\Lambda^2 \otimes \text{Sym}^2$ ):  $\begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \end{array} \otimes \square\square \cong \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & & \\ \hline \end{array} \oplus \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \\ \hline \end{array}$
- **E** ( $\Lambda^2 \otimes \Lambda^2$ ):  $\begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \end{array} \otimes \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \end{array} \cong \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \oplus \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \\ \hline \end{array}$

By solving the linear system to match the target multiplicities, we can identify a strictly positive integer solution directly from this restricted basis:

$$V^{\otimes 4} \cong 1C \oplus 2D \oplus 1E$$

Expanding these selected candidates into their irreducible representations confirms the exact structural balance of the target space. Because all coefficients are positive, the decomposition exists perfectly within the semi-ring of physical vector spaces without the need for virtual subtraction.

**The  $n = 5$  Case: The Necessity of Virtual Representations** As the tensor power increases to  $n = 5$ , the overall space expands to 243 dimensions, spanning five distinct irreducible blocks:

$$(\square)^{\otimes 5} \cong 1 \square\square\square\square\square \oplus 4 \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & & & \\ \hline \end{array} \oplus 5 \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & & \\ \hline \end{array} \oplus 6 \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & & \\ \hline \end{array} \oplus 5 \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array}$$

Extracting the unique non-commuting tensor products yields a new candidate pool:

- **A** ( $\text{Sym}^1 \otimes \text{Sym}^4$ ):  $\square \otimes \square\square\square\square \cong \square\square\square\square\square \oplus \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & & & \\ \hline \end{array}$
- **B** ( $\text{Sym}^2 \otimes \text{Sym}^3$ ):  $\square\square \otimes \square\square\square \cong \square\square\square\square\square \oplus \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & & & \\ \hline \end{array} \oplus \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & & \\ \hline \end{array}$
- **C** ( $\Lambda^3 \otimes \text{Sym}^2$ ):  $\begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} \otimes \square\square \cong \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & & \\ \hline \end{array}$

- **D** ( $\Lambda^2 \otimes \text{Sym}^3$ ):  $\begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \end{array} \otimes \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \end{array} \cong \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & & & \\ \hline \end{array} \oplus \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & & \\ \hline \end{array}$
- **E** ( $\Lambda^2 \otimes \Lambda^3$ ):  $\begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \end{array} \otimes \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} \cong \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array}$

In this instance, fulfilling the competing target multiplicities is mathematically unachievable with purely positive coefficients. To see why, observe the requirements for the  $\begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \end{array}$  and  $\begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array}$  blocks. The target space demands exactly five  $\begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array}$  blocks. Within our candidate pool, Candidate **B** is the only tensor product that produces this shape, strictly forcing its coefficient to be 5.

However, because Candidate **B** jointly produces  $\begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \end{array}$  alongside it, this selection inherently generates five  $\begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \end{array}$  blocks. Our target space requires only one. Because we cannot generate the necessary  $\begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array}$  blocks without simultaneously over-generating  $\begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \end{array}$ , we are forced to subtract the excess. The only way to balance this algebraic system is to introduce a negative coefficient by subtracting four instances of Candidate **A**, successfully mapping the decomposition out of physical space and into the Grothendieck ring:

$$[V^{\otimes 5}] = -4[A] + 5[B] + 3[C] + 3[D] + 5[E]$$

## 2.6 The Representation Ring and the Jacobi-Trudi Identity

As demonstrated by the  $n = 5$  tensor power, reconstructing the target tensor space strictly from a restricted pool of computable building blocks frequently requires negative coefficients. Because physical vector spaces cannot be subtracted, we must shift our mathematical framework from the standard semi-ring of representations into the representation ring to rigorously define and manipulate these algebraic constructions.

**Definition 3** (The Representation Ring / Virtual Representations [5]). *Let  $\text{Irr}(G)$  be the set of isomorphism classes of irreducible representations of a group  $G$ . The representation ring, denoted  $R(G)$ , is the free abelian group generated by these classes. Elements of this ring are virtual representations, taking the form of formal  $\mathbb{Z}$ -linear combinations:*

$$\rho = \sum_{[V_i] \in \text{Irr}(G)} c_i [V_i] \quad (17)$$

where  $c_i \in \mathbb{Z}$ . By allowing negative integer coefficients,  $R(G)$  forms a true ring under direct sum and tensor product, wherein the formal subtraction of modules becomes mathematically well-defined.

**Remark on Notation (Physical Spaces vs. Virtual Representations):** Throughout this text, a strict notational distinction is maintained between actual vector spaces



and elements of the representation ring. When discussing physical modules and invariant subspaces, we use the isomorphism symbol ( $\cong$ ) and the direct sum operator ( $\oplus$ ), for example,  $V \cong U \oplus W$ . However, once the framework shifts into the Grothendieck ring  $R(G)$  to utilize virtual representations, we operate exclusively on equivalence classes. These are denoted by square brackets  $[V]$ , evaluated using standard ring equality ( $=$ ), and manipulated using standard addition ( $+$ ) and subtraction ( $-$ ). For instance, the formal expression  $[V] = [U] - [W]$  is algebraically well-defined within  $R(G)$ .

To isolate the target computationally, we leverage the determinantal formulas for Schur polynomials. Because Schur polynomials map directly to the irreducible representations of  $GL(V)$  and the complete homogeneous symmetric polynomials map to the symmetric powers  $\text{Sym}^k(V)$ , we can map algebraic identities of polynomials directly to virtual constructions within the representation ring.

**Theorem 3** (The Jacobi-Trudi Identity [5]). *For any integer partition  $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_r)$ , the corresponding Schur polynomial  $S_\lambda$  can be expressed as a determinant of complete homogeneous symmetric polynomials  $H_k$ :*

$$S_\lambda = \det(H_{\lambda_i + j - i})_{1 \leq i, j \leq r} \quad (18)$$

where  $H_0 = 1$  and  $H_k = 0$  for  $k < 0$ . Mapped to the representation ring  $R(GL(V))$ , this implies that the equivalence class of any irreducible module  $[V_\lambda]$  is exactly equal to the determinant of an  $r \times r$  matrix whose entries are the symmetric tensor powers  $[\text{Sym}^{\lambda_i + j - i}(V)]$ .

This theorem provides a direct mechanism to construct any target irreducible representation entirely out of symmetric representations. Furthermore, it places a strict upper bound on the complexity of the tensor products required.

**Proposition 1.** *For a  $d$ -dimensional vector space  $V$ , any irreducible polynomial representation of  $GL(V)$  can be explicitly reconstructed in the representation ring using only linear combinations of tensor products of at most  $d-1$  symmetric representations, tensored with a power of the determinant.*

*Proof.* Let  $V_\lambda$  be an irreducible representation indexed by a partition  $\lambda$  with at most  $d$  rows. By factoring out  $m = \lambda_d$  full columns of length  $d$ , we twist the representation by the one-dimensional determinant, yielding  $[V_\lambda] = [V_\mu] \otimes \det^m$ , where  $\mu = (\lambda_1 - \lambda_d, \dots, \lambda_{d-1} - \lambda_d, 0)$ .

Because  $\mu$  has at most  $r \leq d-1$  non-zero parts, applying the Jacobi-Trudi identity to  $[V_\mu]$  yields the determinant of an  $r \times r$  matrix. The Leibniz expansion of this determinant produces a formal sum of products (which correspond to tensor products in

the representation ring) of exactly  $r$  symmetric representations. Since  $r \leq d - 1$ , the Jacobi-Trudi identity not only proves that tensor products of at most  $d - 1$  symmetric powers are sufficient, but its determinantal expansion yields the exact, explicit integer weights required to construct the target space.  $\square$

For our specific computational domain in  $SU(3)$  (where  $d = 3$ ), after factoring out full columns of length three into the determinant, any irreducible representation is indexed by a partition of at most two rows,  $\lambda = (l_1 + l_2, l_2)$ , corresponding to the Dynkin labels  $(l_1, l_2)$ .

Following the proposition above, the Jacobi-Trudi identity for this two-row partition simplifies to a  $2 \times 2$  determinant:

$$S_{(l_1+l_2, l_2)} = \det \begin{pmatrix} H_{l_1+l_2} & H_{l_1+l_2+1} \\ H_{l_2-1} & H_{l_2} \end{pmatrix} = H_{l_1+l_2}H_{l_2} - H_{l_1+l_2+1}H_{l_2-1} \quad (19)$$

Within the representation ring  $R(G)$ , this indicates that the target irreducible module  $V_{(l_1, l_2)}$  can be isolated via a formal subtraction of 2-term symmetric tensor products:

$$[V_{(l_1, l_2)}] = [\text{Sym}^{l_1+l_2}(V) \otimes \text{Sym}^{l_2}(V)] - [\text{Sym}^{l_1+l_2+1}(V) \otimes \text{Sym}^{l_2-1}(V)] \quad (20)$$

where representations with negative indices are defined to be zero. Consequently, any arbitrary block in the tensor space can be algebraically isolated using purely the positive and negative integer weights prescribed deterministically by the Jacobi-Trudi formula, eliminating the need to search for decompositions using combinatorial trees and discarding the necessity for antisymmetric building blocks entirely.

## 2.7 Application to Schatten $p$ -Norms

The ultimate objective of this framework is the efficient evaluation of the weighted Schatten  $p$ -norm of the  $n$ -fold tensor power.

**Definition 4** (Schatten  $p$ -Norm [9]). *Let  $M \in \mathbb{C}^{d \times d}$  be a complex matrix, and let  $\sigma_1(M) \geq \sigma_2(M) \geq \dots \geq \sigma_d(M) \geq 0$  denote its singular values. For any real number  $p \in [1, \infty)$ , the Schatten  $p$ -norm of  $M$  is defined as the standard  $\ell_p$ -norm applied to the vector of its singular values:*

$$\|M\|_p = \left( \sum_{i=1}^d \sigma_i(M)^p \right)^{1/p} \quad (21)$$

*In the limiting case where  $p \rightarrow \infty$ , the Schatten  $\infty$ -norm (often referred to as the spectral or operator norm) is defined as the maximum singular value:*

$$\|M\|_\infty = \max_i \sigma_i(M) = \sigma_1(M) \quad (22)$$

Because the Schatten norm is intrinsically tied to the spectrum of singular values, it is a unitarily invariant matrix norm.

Because the Jacobi-Trudi identity establishes an exact algebraic isomorphism mapping the target irreducible block to a linear combination of computable symmetric products, we completely circumvent the  $\mathcal{O}(3^{2n})$  bottleneck. The Schatten  $p$ -norm raised to the  $p$ -th power,  $\|M\|_p^p$ , is additive over block diagonal matrices. Therefore, when evaluating a linear combination of tensor powers  $M = \sum_i t_i A_i^{\otimes n}$ , the norm contribution of any target block  $V_{(l_1, l_2)}$  can be isolated using the Jacobi-Trudi difference of the norms of the corresponding linearly combined tensor products.

Crucially, because we are summing across multiple matrices  $A_i$ , the respective scalar weights  $t_i$  and the factored determinant powers  $m$  must be applied inside the sum before the spectrum is evaluated. The exact block-wise norm contribution is given by:

$$\left\| \sum_i t_i \det(A_i)^m V_{(l_1, l_2)}(A_i) \right\|_p^p = \left\| \sum_i t_i \det(A_i)^m \left( \pi_{\text{Sym}^{l_1+l_2}}(A_i) \otimes \pi_{\text{Sym}^{l_2}}(A_i) \right) \right\|_p^p - \left\| \sum_i t_i \det(A_i)^m \left( \pi_{\text{Sym}^{l_1+l_2+1}}(A_i) \otimes \pi_{\text{Sym}^{l_2-1}}(A_i) \right) \right\|_p^p \quad (23)$$

The total norm of the  $n$ -fold tensor power linear combination is then directly aggregated by evaluating these combined symmetric tensor products and scaling them by the Hook-length multiplicities  $f^{(l_1, l_2)}$  of their corresponding target diagrams:

$$\left\| \sum_i t_i A_i^{\otimes n} \right\|_p = \left( \sum_{(l_1, l_2)} f^{(l_1, l_2)} \left\| \sum_i t_i \det(A_i)^m V_{(l_1, l_2)}(A_i) \right\|_p^p \right)^{1/p} \quad (24)$$

### 3 Examples

#### 3.1 Illustrative Examples: Tensor Product Decompositions

To demonstrate the utility of this computational framework, we explicitly construct the target representations of the tensor space. We define the  $n$ -th tensor power of the fundamental representation using the equivalence  $V^{\otimes n} \equiv (\square)^{\otimes n}$ . By Schur-Weyl duality, this full tensor space decomposes into a direct sum of Weyl modules, where the multiplicities are given by the dimensions of the corresponding symmetric group representations. Here, we restrict our space to  $SU(3)$ , meaning our Young diagrams are capped at a maximum of three rows.

##### 3.1.1 Combinatorial Evaluation of Target Space via Hook and Weyl Formulas

The structural decomposition of  $V^{\otimes n}$  requires determining two distinct quantities for each valid Young diagram: the representation's multiplicity within the tensor space (via the Hook-Length Formula, Theorem 2) and its internal spatial dimension (via the Weyl Dimension Formula, Eq. 16).

To explicitly demonstrate this dual calculation, consider the target tensor space for  $n = 4$  in  $SU(3)$ . The valid partitions are (4), (3,1), (2,2), and (2,1,1). We populate each cell of the corresponding Young diagrams with its exact calculated hook length:

- **Partition (4):** Translates to Dynkin labels (4, 0).

$$\begin{array}{|c|c|c|c|} \hline 4 & 3 & 2 & 1 \\ \hline \end{array}$$

**Multiplicity:** The hook product is  $4 \times 3 \times 2 \times 1 = 24$ , yielding  $f^{(4)} = \frac{4!}{24} = 1$ .

**Dimension:** Eq. 16 evaluates to  $\frac{1}{2}(5)(1)(6) = 15$ .

- **Partition (3,1):** Translates to Dynkin labels (2, 1).

$$\begin{array}{|c|c|c|} \hline 4 & 2 & 1 \\ \hline 1 & & \\ \hline \end{array}$$

**Multiplicity:** The hook product is  $4 \times 2 \times 1 \times 1 = 8$ , yielding  $f^{(3,1)} = \frac{4!}{8} = 3$ .

**Dimension:** Eq. 16 evaluates to  $\frac{1}{2}(3)(2)(5) = 15$ .

- **Partition (2,2):** Translates to Dynkin labels (0, 2).

$$\begin{array}{|c|c|} \hline 3 & 2 \\ \hline 2 & 1 \\ \hline \end{array}$$

**Multiplicity:** The hook product is  $3 \times 2 \times 2 \times 1 = 12$ , yielding  $f^{(2,2)} = \frac{4!}{12} = 2$ .

**Dimension:** Eq. 16 evaluates to  $\frac{1}{2}(1)(3)(4) = 6$ .

- **Partition** (2,1,1): Translates to Dynkin labels (1, 0).

$$\begin{array}{|c|c|} \hline 4 & 1 \\ \hline 2 & \\ \hline 1 & \\ \hline \end{array}$$

**Multiplicity:** The hook product is  $4 \times 1 \times 2 \times 1 = 8$ , yielding  $f^{(2,1,1)} = \frac{4!}{8} = 3$ .

**Dimension:** Eq. 16 evaluates to  $\frac{1}{2}(2)(1)(3) = 3$ .

By grouping these results, the invariant block structure is fully characterized. We can dynamically verify the total dimensionality of the tensor space:  $(1 \times 15) + (3 \times 15) + (2 \times 6) + (3 \times 3) = 81 = 3^4$ .

### 3.1.2 The $n = 4$ Case: Exact Deterministic Decomposition via Jacobi-Trudi

For the fourth tensor power in  $SU(3)$ , the overall  $3^4 = 81$ -dimensional space decomposes into the target representations:

$$V^{\otimes 4} \cong 1 \begin{array}{|c|c|c|c|} \hline & & & \\ \hline \end{array} \oplus 3 \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \oplus 2 \begin{array}{|c|c|} \hline & \\ \hline & \\ \hline \end{array} \oplus 3 \begin{array}{|c|c|} \hline & \\ \hline & \\ \hline \end{array}$$

Using the Jacobi-Trudi identity, we map this decomposition to our exact virtual representation framework, relying purely on symmetric tensor products  $[\text{Sym}^a \otimes \text{Sym}^b]$ . For each target shape, we apply the determinant formula and scale the result by the target's multiplicity in  $V^{\otimes 4}$ :

- **Partition** (4):  $l_1 = 4, l_2 = 0$ . Multiplicity = 1.  
 $[V_{(4,0)}] = 1 \cdot ([\text{Sym}^4 \otimes \text{Sym}^0] - [\text{Sym}^5 \otimes \text{Sym}^{-1}]) = 1 [\text{Sym}^4 \otimes \text{Sym}^0]$
- **Partition** (3,1):  $l_1 = 2, l_2 = 1$ . Multiplicity = 3.  
 $[V_{(2,1)}] = 3 \cdot ([\text{Sym}^3 \otimes \text{Sym}^1] - [\text{Sym}^4 \otimes \text{Sym}^0])$
- **Partition** (2,2):  $l_1 = 0, l_2 = 2$ . Multiplicity = 2.  
 $[V_{(0,2)}] = 2 \cdot ([\text{Sym}^2 \otimes \text{Sym}^2] - [\text{Sym}^3 \otimes \text{Sym}^1])$
- **Partition** (2,1,1): Reduces to (1) with  $\det^1$ .  $l_1 = 1, l_2 = 0, m = 1$ . Multiplicity = 3.  
 $[V_{(1,0)} \det] = 3 \cdot ([\text{Sym}^1 \otimes \text{Sym}^0] \det - 0) = 3 [\text{Sym}^1 \otimes \text{Sym}^0] \det$

Summing these terms algebraically yields the complete, exact block decomposition in the representation ring:

$$[V^{\otimes 4}] = -2 [\text{Sym}^4] + 1 [\text{Sym}^3 \otimes \text{Sym}^1] + 2 [\text{Sym}^2 \otimes \text{Sym}^2] + 3 [\text{Sym}^1] \det$$

Expressed explicitly with Young diagrams, this virtual decomposition takes the visual form within the Grothendieck ring:

$$\begin{aligned} [V^{\otimes 4}] &= -2 [\square\square\square\square] + 1 [\square\square\square \otimes \square] + 2 [\square\square \otimes \square\square] + 3 \left[ \square \otimes \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \end{array} \right] \\ &= 1 [\square\square\square\square] + 3 \left[ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \right] + 2 \left[ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \right] + 3 \left[ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \right] \end{aligned}$$

Expanding the tensor products via Pieri's formulas confirms that this virtual combination indeed perfectly reconstructs the original target space.

Note that the representation  $\begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \end{array}$  is exactly the determinant representation, i.e. the representation  $\begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array}$  is equivalent to  $\square$  with a determinant twist (up to isomorphism).

### 3.2 Explicit Evaluation of the Multinomial Expansion

To demonstrate the mathematical mechanics of the closed-form summation derived in Equation (14), we explicitly evaluate the transition amplitudes for the second symmetric power ( $k = 2$ ) of an arbitrary  $3 \times 3$  matrix ( $d = 3$ ).

Let  $M \in \text{GL}(3, \mathbb{C})$  be parameterized as:

$$M = \begin{pmatrix} M_{1,1} & M_{1,2} & M_{1,3} \\ M_{2,1} & M_{2,2} & M_{2,3} \\ M_{3,1} & M_{3,2} & M_{3,3} \end{pmatrix}$$

**Basis Generation and Normalization** The orthonormal basis states are indexed by weak compositions of  $k = 2$  into  $d = 3$  parts. The normalization factor for each state  $\mathbf{v} = (v_1, v_2, v_3)$  is governed by Equation (9). For instance, consider the target state  $|\mathbf{r}\rangle = |2, 0, 0\rangle$  and the source state  $|\mathbf{c}\rangle = |1, 1, 0\rangle$ . Their respective normalizations are:

$$\mathcal{N}(\mathbf{r}) = \sqrt{\frac{2!0!0!}{2!}} = 1, \quad \mathcal{N}(\mathbf{c}) = \sqrt{\frac{1!1!0!}{2!}} = \frac{1}{\sqrt{2}}$$

**Transition Evaluation via Exponent Matrices** To evaluate the matrix element  $\langle 2, 0, 0 | \pi_{\text{Sym}^2}(M) | 1, 1, 0 \rangle$ , we must identify all valid  $3 \times 3$  non-negative integer exponent matrices  $E$  that satisfy the marginal constraints (Equation 10). The row sums must equal  $\mathbf{r} = (2, 0, 0)$  and the column sums must equal  $\mathbf{c} = (1, 1, 0)$ .

There is exactly one matrix  $E$  satisfying these boundary conditions:

$$E = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Applying the closed-form expansion (Equation 14), we compute the combinatorial coefficient and the evaluated matrix element:

$$\langle 2, 0, 0 | \pi_{\text{Sym}^2}(M) | 1, 1, 0 \rangle = \left( \frac{\sqrt{2!0!0!1!1!0!}}{1!1!0! \dots 0!} \right) M_{1,1}^1 M_{1,2}^1 = \sqrt{2} M_{1,1} M_{1,2}$$

**Superposition of Transition Paths** The combinatorial nature of the expansion is most evident in diagonal or heavily mixed transitions. Consider the element  $\langle 1, 1, 0 | \pi_{\text{Sym}^2}(M) | 1, 1, 0 \rangle$ . The marginal constraints now require both row and column sums to equal  $(1, 1, 0)$ . This yields exactly two valid exponent configurations:

$$E_1 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} \quad \text{and} \quad E_2 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Each matrix represents a distinct combinatorial path mapping the source to the target. Evaluating the summand for  $E_1$  yields the unpermuted transition  $M_{1,1}M_{2,2}$ , while  $E_2$  yields the permuted transition  $M_{1,2}M_{2,1}$ . Both share the same normalization prefactor of  $\frac{1}{2}$  and a multinomial coefficient of  $2!/(1!1!) = 2$ . Summing over all valid  $E$  provides the exact matrix element:

$$\langle 1, 1, 0 | \pi_{\text{Sym}^2}(M) | 1, 1, 0 \rangle = M_{1,1}M_{2,2} + M_{1,2}M_{2,1}$$

By systematically iterating over all valid exponent matrices  $E$  for every pair of basis states, the full invariant block  $\pi_{\text{Sym}^k}(M)$  is analytically populated. This circumvents the exponential scaling bottleneck entirely, constructing the polynomial representations without ever generating or reducing the tensor space  $V^{\otimes k}$ .

## 4 Computational Methodology and Software Architecture

The theoretical framework established in the preceding sections demonstrates that any polynomial property of  $V^{\otimes n}$  can be exactly computed via linear combinations of the invariant blocks of 2-term symmetric tensor products (for  $V = \mathbb{C}^3$ ). However, practically executing this over thousands of large matrices requires an optimized software architecture.

The algorithmic pipeline of our program is divided into three distinct phases: (1) the offline pre-computation and compression of representation matrices, (2) the deterministic structural decomposition via Jacobi-Trudi, and (3) the online vectorization and aggregation of the target matrix function (such as the Schatten  $p$ -norm).

### 4.1 High-Level Algorithmic Pipeline

**Phase 1: Pre-computation of Invariant Blocks.** The foundation of the algorithm relies on the matrix representations of the symmetric powers  $\text{Sym}^k(\mathbb{C}^3)$ . Because the analytical structures of these representations are independent of any specific numerical input matrix, they are generated purely symbolically once. The program iteratively constructs these algebraic matrices for all required values of  $k$ .

**Phase 2: Deterministic Structural Decomposition.** When queried to evaluate a tensor power  $n$ , the program formulates the block decomposition directly using the Jacobi-Trudi formulas. It generates the integer partitions, calculates their standard tableaux multiplicities, and directly maps them to the exact signed weights corresponding to the 2-term symmetric tensor products.

**Phase 3: Rapid Evaluation and Aggregation.** For a specified linear combination of numerical input matrices, the program loads the necessary pre-computed representations and evaluates them dynamically. It constructs the tensor product blocks and scales each internally by its respective user-defined linear coefficient and the structural determinant factor  $\det(M)^m$ . Once these weighted composite blocks are formed, the target operator metric is computed, such as evaluating the singular value decomposition (SVD) to determine the block-wise Schatten  $p$ -norm. Finally, these individual block metrics are aggregated across the virtual representation, scaled strictly by the exact representation multiplicities calculated in Phase 2.



## 4.2 Data Architecture: Sparse Tensor Compression

A naive implementation of Phase 1 would involve saving large arrays of symbolic polynomials to disk, and evaluating them in Phase 3 using symbolic substitution. This is computationally expensive; evaluating thousands of string-based polynomials at runtime introduces severe latency. Furthermore, storing the full symbolic matrices with strings is memory-inefficient.

To achieve superior runtime and memory efficiency, we developed a specialized data architecture that reduces symbolic polynomial evaluation to a single Sparse Matrix-Vector Multiplication (SpMV).

Let  $N_k$  be the dimension of the representation  $\text{Sym}^k(\mathbb{C}^3)$ . The representation matrix  $\pi_{\text{Sym}^k}(M)$  is an  $N_k \times N_k$  matrix. Every entry in this matrix is a homogeneous polynomial of degree  $k$ , formed from the 9 variables of the  $3 \times 3$  input matrix  $M$  (flattened row-wise as  $M_{1,1}, M_{1,2}, \dots, M_{3,3}$ ).

Let  $E_{\text{table}}$  be the ordered set of all valid non-negative integer exponent configurations  $\mathbf{e} = (e_1, \dots, e_9)$  such that  $\sum e_i = k$ . The total number of unique monomials is given by the combination with repetition:  $|E_{\text{table}}| = \binom{k+9-1}{9-1}$ .

Crucially, the elements of  $E_{\text{table}}$  are generated and indexed according to a strict *reverse lexicographic order*. This fixed ordering enforces a deterministic bijection between the abstract algebraic monomials and the physical column indices of our data structures. Without a strictly preserved basis order, the numerical vectors cannot be reliably reconstructed during the dot product operations. For any numerical evaluation, the evaluated values of these ordered monomials are stored in a column vector  $\vec{\mu} \in \mathbb{C}^{|E_{\text{table}}|}$ .

Because the matrix elements are purely linear combinations of these monomials, the  $N_k \times N_k$  symbolic matrix can be completely decoupled into two static, numerical data structures:

1. **The Exponent Table ( $E_{\text{table}}$ ):** A dense unsigned integer array of size  $|E_{\text{table}}| \times 9$ . The  $j$ -th row stores the exact reverse lexicographically ordered exponent configuration  $\mathbf{e}_j$  required to compute the  $j$ -th monomial.
2. **The Sparse Coefficient Matrix ( $T$ ):** A highly sparse, two-dimensional coefficient matrix  $T \in \mathbb{R}^{N_k^2 \times |E_{\text{table}}|}$ . Let  $\vec{\pi}$  be the flattened  $N_k^2 \times 1$  vector of the target representation matrix. The matrix  $T$  encodes the exact analytical coefficients such that the representation can be recovered via the linear map  $\vec{\pi} = T\vec{\mu}$ . Specifically, the entry  $T_{i,j}$  stores the precise floating-point scalar coefficient of the  $j$ -th ordered monomial present within the  $i$ -th flattened matrix element.

These structures are compressed and serialized to disk. By stripping out all symbolic overhead, this step is performed exactly once per dimension  $k$ , forever reducing the evaluation of polynomial matrix elements to highly optimized numerical linear algebra.

### 4.3 Runtime Vectorization and Parallelization

By organizing the data into this sparse architecture, the runtime evaluation of  $\pi_{\text{Sym}^k}(M)$  for a numeric matrix  $M$  is stripped of all symbolic overhead. The evaluation algorithm proceeds as follows:

1. The 9 numerical elements of  $M$  are flattened into a 1D vector.
2. Using the static Exponent Table, the monomial vector  $\vec{\mu}$  is computed globally via highly optimized, vectorized element-wise power and product operations natively in C (via NumPy).
3. The evaluated flattened matrix is obtained via a single linear contraction:  $\vec{\pi}_{\text{flat}} = T\vec{\mu}$ .
4. The vector  $\vec{\pi}_{\text{flat}}$  is dynamically reshaped back into the  $N_k \times N_k$  matrix structure.

This completely eliminates loops during evaluation, making this step very efficient.

Furthermore, Phase 1 (the symbolic generation and sparse factorization of these representations) is a highly parallelizable problem. Because the generation of  $\text{Sym}^k$  is completely independent of  $\text{Sym}^{k'}$ , the program implements a multiprocessing pool to distribute the polynomial expansion tasks across all available CPU cores. As  $k$  increases, the number of basis states and valid transition matrices grows factorially, making this concurrent offline calculation critical for extending the framework to compute large tensor powers.

### 4.4 Algorithmic Efficiency of the Jacobi-Trudi Decomposition

The framework utilizes the Jacobi-Trudi identity to compute the signed integer coefficients purely deterministically (see Section 2.6).

Let  $N_{\text{targets}}$  be the number of unique target irreducible representations present in the  $n$ -th tensor power. Because the Jacobi-Trudi formula explicitly defines the basis transformation from arbitrary irreducible modules to symmetric tensor products, the integer coefficient vector mapping the tensor space to computable blocks is generated in  $\mathcal{O}(N_{\text{targets}})$  operations.

**Eliminating Search Overhead** By abandoning feasibility searches, we entirely remove the need for external linear solvers and complex combinatorial checks. The computational overhead of Phase 2 is reduced to near zero, as the coefficients are yielded instantly via the combinatorial equations. Furthermore, because the Jacobi-Trudi identity guarantees construction using strictly symmetric representations, we can discard the antisymmetric tensor block  $\Lambda^2(\mathbb{C}^3)$ . However, since it’s only 3 dimensional, it *could* be included as a candidate block in an ILP formulation to potentially find a solution that then runs faster for evaluation, albeit at the cost of introducing search overhead.

## 4.5 Alternative Approach and Future Work: Integer Linear Programming

In earlier iterations of this computational framework, the structural decomposition of the tensor space was resolved dynamically using Integer Linear Programming (ILP). While the current methodology relies on the Jacobi-Trudi identities, which provide an exact, mathematically proven, and deterministic combinatorial generation of the virtual multiplicities, the ILP approach remains a highly promising avenue for future algorithmic optimization.

The ILP algorithm formulates the decomposition by searching for a multiplicity vector  $\mathbf{x} \in \mathbb{Z}^{N_{\text{cand}}}$  that maps the target tensor space to a pre-calculated pool of 2-term candidate blocks, solving the linear system within the representation ring:  $\sum x_i [\text{Candidate}_i] = [V^{\otimes n}]$ . For the candidates, we can include the antisymmetric representation  $\Lambda^2(\mathbb{C}^3)$  for a potentially quicker solution. As demonstrated previously during our theoretical exploration of the  $n = 4$  and  $n = 5$  tensor powers (see Section 2.5), this search space relies on satisfying competing target multiplicities. While lower powers may yield strictly positive solutions, the solver seamlessly maps complex combinations into the Grothendieck ring by naturally evaluating the required negative coefficients.

**Future Work: Cost Optimization via ILP** Because the linear system mapping candidates to target multiplicities is heavily underdetermined ( $N_{\text{cand}} \gg N_{\text{targets}}$ ), there exist infinitely many valid integer vectors  $\mathbf{x}$  that satisfy the target space. The current Jacobi-Trudi implementation yields exactly one deterministic, mathematically proven valid expansion.

However, transitioning the ILP from a pure feasibility search to an optimization problem presents an opportunity for cost optimization. By introducing an objective function designed to minimize downstream computational costs, for example, penalizing the selection of candidate blocks possessing bigger spatial matrix dimensions, an ILP solver could

discover a solution that reduces the  $\mathcal{O}(N^3)$  Singular Value Decomposition bottleneck. This would introduce longer runtimes during Phase 2, but if there is performance to gain in Phase 3, the ILP solutions too could be saved, and therefore would only need to be generated once per  $n$  (and  $d$ ).

While this theoretical performance gain requires rigorous benchmarking against the Jacobi-Trudi method, equipping the ILP with a runtime-minimizing cost function remains an objective for future research.

## 5 Results

### 5.1 Dimensionality and Memory Space Scaling

The fundamental limitation of computing large tensor powers directly is the exponential explosion of the matrix dimension. For a  $3 \times 3$  input matrix, the spatial dimension of the full tensor space  $V^{\otimes n}$  scales strictly as  $3^n$ . Under explicit Kronecker expansion, this unmitigated exponential scaling rapidly exhausts available system memory, creating a severe operational bottleneck.

In contrast, our framework restricts all computations to structurally decomposed invariant blocks. By leveraging the Jacobi-Trudi identity (as outlined in Section 2.6), the algorithm evaluates target spaces exclusively via the tensor product of two symmetric representations,  $\text{Sym}^a(\mathbb{C}^3) \otimes \text{Sym}^b(\mathbb{C}^3)$ .

By the Weyl Dimension Formula (Equation 16), the dimension of a single symmetric representation  $\text{Sym}^k(\mathbb{C}^3)$  is exactly  $\frac{1}{2}(k+1)(k+2)$ . The dimension of the composite candidate block is therefore the product of the dimensions of its symmetric factors. Because the total tensor power  $n$  dictates the maximum number of boxes in the corresponding Young diagrams, the representation labels are bounded by the constraint  $a + b \leq n$ .

To determine the absolute maximum spatial dimension generated by the algorithm, we maximize this product subject to  $a + b = n$ . The product is strictly maximized when the parameters are distributed as evenly as possible, yielding  $a \approx n/2$  and  $b \approx n/2$ . Substituting this into the combined dimension formula provides the explicit sub-exponential limit:

$$\max \dim \left( \text{Sym}^a(\mathbb{C}^3) \otimes \text{Sym}^b(\mathbb{C}^3) \right) \approx \left[ \frac{1}{2} \left( \frac{n}{2} + 1 \right) \left( \frac{n}{2} + 2 \right) \right]^2 = \mathcal{O}(n^4)$$

Crucially, we are not restricted to estimating via asymptotic upper bounds. Because the Jacobi-Trudi formula explicitly prescribes the exact linear combination of symmetric tensor products required to resolve the target representation, we know the exact blocks we need to evaluate.

As implemented in the algorithmic solver, for a given tensor power  $n$ , the framework generates all valid integer partitions  $\lambda = (\lambda_1, \lambda_2, \lambda_3) \vdash n$ . Each partition is mapped to its Dynkin labels  $l_1 = \lambda_1 - \lambda_2$  and  $l_2 = \lambda_2 - \lambda_3$ , dictating the exact virtual decomposition:

$$[V_{(l_1, l_2)}] = [\text{Sym}^{l_1+l_2}(\mathbb{C}^3) \otimes \text{Sym}^{l_2}(\mathbb{C}^3)] - [\text{Sym}^{l_1+l_2+1}(\mathbb{C}^3) \otimes \text{Sym}^{l_2-1}(\mathbb{C}^3)] \quad (25)$$

Letting  $D(k) = \frac{1}{2}(k+1)(k+2)$  denote the spatial dimension of  $\text{Sym}^k(\mathbb{C}^3)$ , the explicit matrix dimensions of the two candidate blocks required to evaluate this target are exactly:

$$\dim(\text{Block}_1) = D(l_1 + l_2) \cdot D(l_2) = \frac{1}{4}(l_1 + l_2 + 1)(l_1 + l_2 + 2)(l_2 + 1)(l_2 + 2)$$

and, provided the negative term is valid ( $l_2 \geq 1$ ):

$$\dim(\text{Block}_2) = D(l_1 + l_2 + 1) \cdot D(l_2 - 1) = \frac{1}{4}(l_1 + l_2 + 2)(l_1 + l_2 + 3)(l_2)(l_2 + 1)$$

Furthermore, because the total tensor space  $V^{\otimes n}$  is a direct sum over all target multiplicities, the complete Jacobi-Trudi expansion can be structurally simplified before any numerical matrices are instantiated. Within the representation ring, the positive and negative symmetric tensor products stemming from different target representations often overlap. By accumulating these terms globally-summing their respective standard tableaux multiplicities-redundant blocks algebraically combine, and opposing terms cancel.

By iterating this explicit mapping over all valid partitions and dynamically simplifying the direct sum, the spatial dimension of every unique matrix required for the structural decomposition is deterministically known for any given  $n$ . This exact knowledge bypasses combinatorial search entirely and explicitly maps the memory required to escape the direct tensor product bottlenecks.

To empirically validate this theoretical memory reduction, Figure 1 illustrates the comparative memory footprint required to store the resulting matrix representations. Storing a dense  $N \times N$  matrix of complex double-precision floating-point numbers requires  $128 \times N^2$  bits. For the brute-force explicit Kronecker construction,  $N = 3^n$ , leading to a strict  $\mathcal{O}(9^n)$  space requirement. As shown, the memory required for the direct tensor product exceeds standard computational limits (e.g., 100 Gigabytes) well before  $n = 15$ . In contrast, the block decomposition representation-aggregating only the unique, maximally compressed symmetric tensor products-remains strictly polynomial, utilizing a fraction of the memory even at  $n = 26$ .

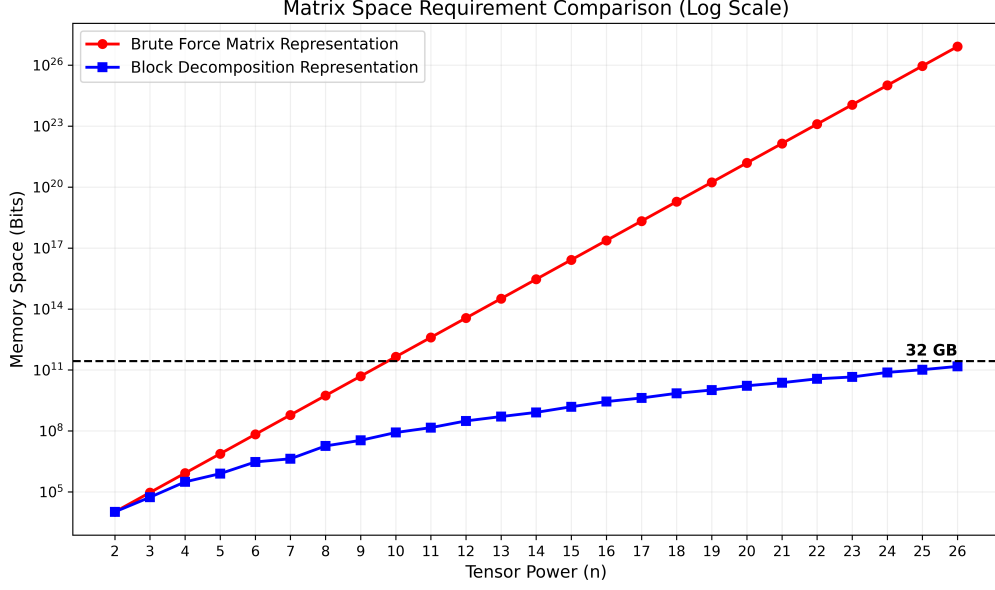


Figure 1: Comparison of the total matrix memory space requirement (in bits) between the explicit Kronecker product and the block decomposition representation for a single matrix. The brute-force method scales exponentially, while the algebraic reduction remains within standard hardware capacities.

Furthermore, Figure 2 explicitly traces the dimensional components that drive this polynomial memory consumption. The maximum spatial dimension of any single evaluated candidate block (the orange trajectory) scales as  $\mathcal{O}(n^4)$ , drastically diverging from the exponential dimension of the full tensor space  $3^n$  (the red dashed trajectory). Because the SVD algorithm acts exclusively on these isolated sub-blocks, capping the maximum block dimension is the primary driver of the algorithm’s operational viability, that is, the bottleneck will always be the largest block.

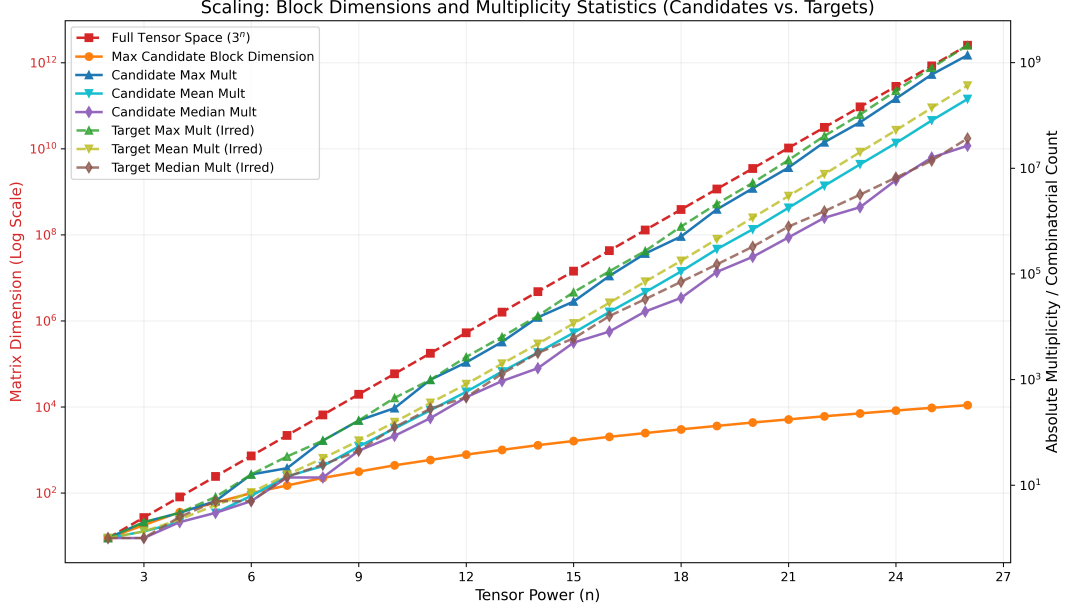


Figure 2: Empirical scaling of block dimensions and virtual multiplicities. The graph highlights the sub-exponential bounding of the maximum candidate block dimension against the unmitigated exponential growth of the full tensor space. Note that the Full Tensor Space (red) and Max Candidate Block Dimension (orange) are on the left scale, while the others are on the right scale.

**Remark (Space vs Time bottleneck):** Figure 1 shows the total memory (in bits) that storing the *full* blockdiagonal representation takes. One of the implemented functions gives these matrices in the diagonal back with their respective multiplicities, so that other calculations can be made, e.g. (log)determinant. However, in the case of calculating the Schatten  $p$ -norm, only one block at a time is actually in memory, which drastically reduces the memory needed to run the algorithm.

## 5.2 Computational Runtime Scaling

To evaluate the practical computational runtime<sup>1</sup>, the brute-force and proposed algorithms were benchmarked by calculating both the structural matrices and the Schatten 1-norm of the linear combination  $\frac{1}{2}A^{\otimes n} - \frac{1}{2}B^{\otimes n}$ , where  $A$  and  $B$  are randomly generated  $3 \times 3$  complex density matrices. This specific expression was chosen as it mirrors the exact matrix norm required to compute the minimum error probability in symmetric asymptotic quantum binary hypothesis testing, representing a use case for the framework.

<sup>1</sup>All empirical benchmarks presented in this section were executed on a workstation equipped with an Intel Core i5-14600K processor and 32GB of DDR5 RAM.



Before measuring any complex matrix operations, we must consider the baseline time required simply to construct the representation matrices in memory. Figure 3 captures this initial overhead. The explicit Kronecker product requires calculating and allocating  $3^{2n}$  matrix elements. As empirical benchmarks demonstrate, this purely structural generation hits the exponential wall rapidly, taking over two seconds for  $n = 9$ . Conversely, the proposed framework-leveraging the pre-computed sparse exponent tables described in Section 4.2-evaluates the structural blocks analytically, bypassing the exponential allocation bottleneck and processing the  $n = 26$  case in under a minute. Note that for  $n \leq 5$  the Brute-force method is faster, because our framework involves opening the pre-saved files and loading the numbers into memory, which has it’s own overhead (hence the extra 1-2 milliseconds).

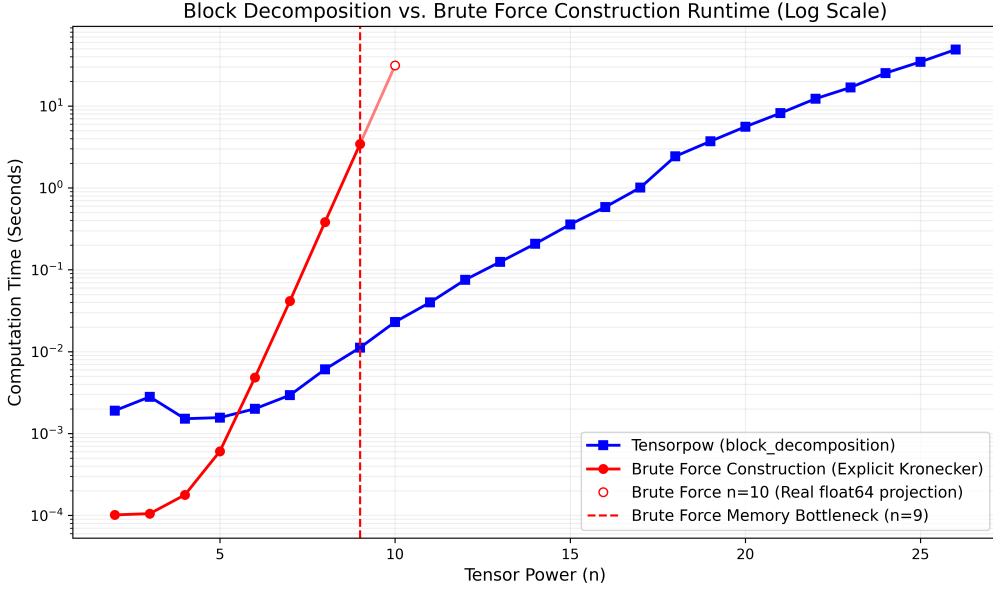


Figure 3: Runtime comparison for the baseline construction of the representation matrices (logarithmic scale). The explicit Kronecker approach succumbs to the exponential allocation bottleneck after  $n = 9$ , the  $n = 10$  measurement was made on a real valued matrix, as an estimation.

The true computational bottleneck, however, emerges during the evaluation of the matrix norm. The Schatten  $p$ -norm requires the computation of the spectrum of singular values. The Singular Value Decomposition (SVD) possesses an asymptotic time complexity of  $\mathcal{O}(N^3)$  for an  $N \times N$  matrix.

For the brute-force methodology, the algorithm must evaluate the SVD on the full  $3^n \times 3^n$  constructed tensor space. The computational runtime therefore scales as:

$$\mathcal{O}\left((3^n)^3\right) = \mathcal{O}(3^{3n})$$

This unmitigated  $\mathcal{O}(3^{3n})$  scaling renders the operation computationally intractable almost immediately.

Conversely, our representation-theoretic algorithm processes the singular values block-wise. The maximum matrix dimension processed by any individual SVD operation is bounded by the size of the largest symmetric tensor product block, which scales as  $\mathcal{O}(n^4)$  (as verified in Figure 2). The runtime complexity for evaluating the SVD on this maximum block is significantly reduced to  $\mathcal{O}((n^4)^3) = \mathcal{O}(n^{12})$ .

To establish a rigorous upper bound for the total algorithmic runtime, we must account for the total number of composite blocks generated by the explicit Jacobi-Trudi sum. The number of valid partitions for  $SU(3)$  scales as  $\mathcal{O}(n^2)$ . Therefore, the total computational runtime of the proposed algorithm is asymptotically bounded by the product of the total number of evaluated blocks and the maximum individual block SVD complexity:

$$\text{Total Runtime} = \mathcal{O}(n^2) \times \mathcal{O}(n^{12}) = \mathcal{O}(n^{14})$$

While  $\mathcal{O}(n^{14})$  is a high-degree polynomial, it represents an algebraic reduction from exponential complexity. Figure 4 explicitly demonstrates the physical impact of this theoretical reduction.

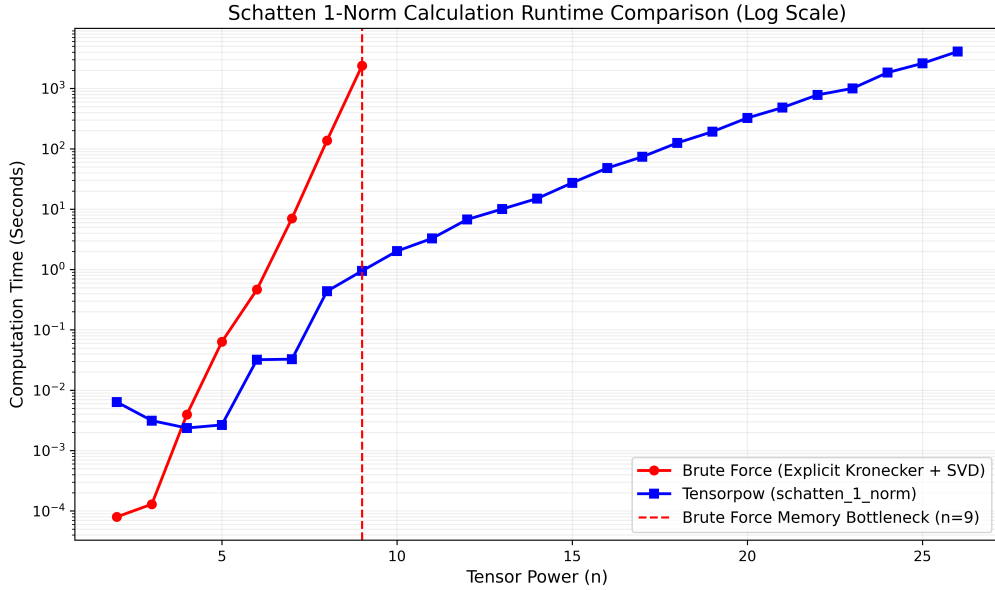


Figure 4: Total calculation runtime for the Schatten 1-norm (logarithmic scale). The  $\mathcal{O}(3^{3n})$  bottleneck halts the brute-force approach at  $n = 9$ , whereas the representation-theoretic framework successfully processes  $n = 26$  in comparable execution times.

At  $n = 9$ , the brute-force SVD evaluation requires thousands of seconds to resolve. In contrast, by confining the  $\mathcal{O}(N^3)$  SVD bottleneck strictly to a polynomially bounded

number of polynomially bounded sub-blocks, the block decomposition framework successfully evaluates tensor powers up to  $n = 26$  within similar computational timeframes, entirely circumventing the exponential scaling barrier. Note that in this case, even  $n = 4$  is slightly faster for the blockdiagonal evaluation.

## 6 Software Implementation and Open-Source Package

To ensure the theoretical framework developed in this thesis is practically accessible for applications in quantum information theory and other fields, the algorithms have been packaged and published as an open-source Python library, `tensorpow` [10]. The full source code, including the deterministic Jacobi-Trudi solver, the sparse tensor compression architecture and the mentioned functions (see Section 6.3) is publicly available on GitHub [11].

### 6.1 Package Capabilities and Storage Constraints

For  $3 \times 3$  matrices, the framework evaluates the exact virtual representation decomposition via the Grothendieck ring. While the algorithm has been successfully tested and executed on local hardware for tensor powers up to  $n = 30$ , the version of `tensorpow` currently distributed via the Python Package Index (PyPI) ships with pre-computed sparse representation tables supporting tensor powers up to  $n = 26$ . This artificial cap is strictly a constraint dictated by PyPI’s maximum package wheel- and GitHub’s file size limits. Future extensions of the package may include on-demand remote fetching to support higher tensor powers without inflating the base installation size.

### 6.2 Integration of $2 \times 2$ Matrix Decompositions

In addition to the novel  $SU(3)$  methodology, the `tensorpow` package seamlessly supports operations on  $2 \times 2$  matrices. The foundational representation theory and algebraic decomposition for the  $d = 2$  case were originally developed and implemented in Mathematica by Weiner and Nietert [12]. For this thesis, that mathematical architecture was translated into Python and fully integrated into the `TensorPowerCalculator` class pipeline. Because the  $SU(2)$  irreps are all symmetric, there is no need for building up the target representation with Jacobi-Trudi (or an ILP solver). Similarly, there is no need for virtual representations (and the Grothendieck ring), we can stay in the usual vector

space. The `tensorpow` package ships with native support for  $2 \times 2$  matrix evaluations up to  $n = 80$ .

### 6.3 Concurrency and Parallelization Strategy

An important design choice in the software architecture is that the core evaluation methods, specifically `block_decomposition` and `schatten_p_norm_weighted`, are implemented as single-threaded processes. While it is theoretically possible to parallelize the internal Singular Value Decompositions (SVDs) since the blocks are algebraically independent, the library intentionally avoids internal multi-threading.

In practice, high-performance computing tasks involving large tensor powers rarely require evaluating a single matrix configuration. Instead, researchers typically need to evaluate the Schatten  $p$ -norm across a massive array of different matrices, linear coefficients, or tensor powers. By keeping the block-level evaluations single-threaded, the library relies on the end-user to implement (parallel) outer loops (e.g., using `multiprocessing` or cluster arrays). This approach prevents the thread contention and overhead that occurs when nested linear algebra libraries attempt to spawn threads simultaneously, resulting in better CPU utilization and overall throughput for large-scale experiments.

## 7 Conclusion

The evaluation of high tensor powers of matrices is a fundamental operation in quantum hypothesis testing. However, direct algebraic approaches inevitably encounter an exponential memory and processing bottleneck. For a  $d \times d$  matrix raised to the  $n$ -th tensor power, the spatial dimension scales as  $d^n$ , driving the  $\mathcal{O}(N^3)$  computational complexity of the Singular Value Decomposition to  $\mathcal{O}(d^{3n})$ .

In this thesis, we developed and implemented a representation-theoretic framework to bypass this exponential wall. By leveraging Schur-Weyl duality, we recognized that the invariant structures of the tensor space could be evaluated independently. Because explicitly computing highest-weight irreducible representations is computationally prohibitive, we shifted our framework into the Grothendieck ring. By utilizing the Jacobi-Trudi identity, we mathematically proved that any invariant block within the tensor space can be algebraically reconstructed using strictly virtual combinations (additions and formal subtractions) of easily computable symmetric tensor products.

By decoupling the problem from the exponential tensor space and mapping it to a deterministic  $\mathcal{O}(n^2)$  combinatorial count of sub-exponential blocks, we bounded the maximum individual matrix dimension to  $\mathcal{O}(n^4)$ . This shifted the theoretical worst-case runtime complexity from  $\mathcal{O}(3^{3n})$  to a strictly polynomial  $\mathcal{O}(n^{14})$ . Furthermore, empirical benchmarks demonstrated that favorable block size distributions and algebraic cancellations within the representation ring reduce the effective operational scaling to roughly  $\mathcal{O}(n^7)$  (for our interval of computable  $n$ -s).

The resulting software implementation, **tensorpow**, successfully computes exact properties of tensor powers that are entirely impossible to evaluate via brute-force Kronecker expansion, moving the operational limit on standard hardware from  $n = 9$  to  $n = 26$  and beyond. By open-sourcing this architecture, it is our hope that this algebraic reduction will enable researchers to explore asymptotic behaviors in quantum information theory deeper than previously computationally feasible.

## Acknowledgements

I would like to express my sincere gratitude to my supervisor, Mihály Weiner, for his invaluable support and mentorship throughout the development and writing of this thesis. I am especially thankful for his remarkable ability to break down and explain highly complex mathematical concepts in a clear and practical way, which made this entire project possible.

Finally, I would like to thank my family for their continuous support, patience, and encouragement throughout my studies.

## References

- [1] C. W. Helstrom, *Quantum Detection and Estimation Theory*. New York: Academic Press, 1976.
- [2] A. S. Holevo, “Statistical decision theory for quantum systems,” *Journal of Multivariate Analysis*, vol. 3, no. 4, pp. 337–394, 1973.
- [3] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary. Cambridge University Press, 2010.
- [4] M. M. Wilde, *Quantum Information Theory*. Cambridge University Press, 2013.
- [5] W. Fulton and J. Harris, *Representation Theory: A First Course* (Graduate Texts in Mathematics). Springer New York, 1991, vol. 129. DOI: 10.1007/978-1-4612-0979-9
- [6] R. Goodman and N. R. Wallach, *Symmetry, Representations, and Invariants* (Graduate Texts in Mathematics). Springer New York, 2009, vol. 255. DOI: 10.1007/978-0-387-79852-3
- [7] H. Minc, *Permanents* (Encyclopedia of Mathematics and its Applications). Cambridge University Press, 1984.
- [8] S. Aaronson and A. Arkhipov, “The computational complexity of linear optics,” in *Proceedings of the Forty-Third Annual ACM Symposium on Theory of Computing*, ser. STOC ’11, San Jose, California, USA: Association for Computing Machinery, 2011, pp. 333–342, ISBN: 9781450306911. DOI: 10.1145/1993636.1993682 [Online]. Available: <https://doi.org/10.1145/1993636.1993682>
- [9] R. Bhatia, *Matrix Analysis* (Graduate Texts in Mathematics). Springer New York, 1997, vol. 169. DOI: 10.1007/978-1-4612-0653-8
- [10] M. Á. Juhász, *tensorpow: Python package for computations involving large tensor powers of matrices*, <https://pypi.org/project/tensorpow/>, Python Package Index (PyPI), 2026.
- [11] M. Á. Juhász, *tensorpow source code repository*, <https://github.com/juhaszmartin/tensorpow>, GitHub Repository, 2026.
- [12] M. Weiner, *Tensor powers of  $2 \times 2$  matrices (mathematica implementation)*, [https://github.com/Elektrubadur/tensor-powers\\_of\\_2x2\\_matrices](https://github.com/Elektrubadur/tensor-powers_of_2x2_matrices), GitHub Repository, 2025.